

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=0.8" />
  <title>Think - VEXATRON™</title>
  <link href="https://fonts.googleapis.com/css2?family=Share+Tech+Mono&display=swap" rel="stylesheet" />
  <link rel="stylesheet" href="style.css" />
</style>
  body {
    font-family: 'Share Tech Mono', 'Courier New', monospace;
    background: #0a0a14;
    margin: 0;
    padding: 0;
    transition: background 0.4s, color 0.4s, border-color 0.4s;
  }

  body.theme-green { --primary: #00ff88; --secondary: #00ff88; --bg: #0a1a0a; }
  body.theme-blue { --primary: #4488ff; --secondary: #4488ff; --bg: #0a0a1a; }
  body.theme-grey { --primary: #8899aa; --secondary: #8899aa; --bg: #141414; }
  body.theme-white { --primary: #cccccc; --secondary: #cccccc; --bg: #1a1a1a; color: #eeeeee; }
  body.theme-dark { --primary: #00ffcc; --secondary: #00ffcc; --bg: #0a0a14; }

  header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    padding: 16px 30px;
    background: rgba(10, 10, 20, 0.85);
    border-bottom: 1px solid rgba(0, 255, 204, 0.2);
    transition: border-color 0.4s;
  }
  .logo a { display: inline-block; }
  nav ul {
    list-style: none;
    display: flex;
    gap: 24px;
    margin: 0;
    padding: 0;
  }
  nav ul li a {
    color: #8899aa;
    text-decoration: none;
    font-size: 14px;
    letter-spacing: 1px;
    transition: color 0.3s;
  }
  nav ul li a:hover {
    color: var(--primary, #00ffcc);
    text-shadow: 0 0 8px var(--primary, #00ffcc);
  }

  .content-page {
    min-height: calc(100vh - 90px);
    padding: 20px;
    display: flex;
    justify-content: center;
```

```
    align-items: flex-start;
    padding-top: 40px;
}

.think-container {
    max-width: 720px;
    width: 100%;
    background: rgba(10, 10, 20, 0.85);
    border: 1px solid var(--secondary, #00ffcc);
    box-shadow: 0 0 30px rgba(0, 255, 204, 0.15);
    padding: 40px;
    border-radius: 4px;
    position: relative;
}

.think-container h1 {
    font-size: 2.2rem;
    color: var(--primary, #00ff88);
    text-shadow: 0 0 15px rgba(0, 255, 136, 0.3);
    margin-bottom: 4px;
    letter-spacing: 2px;
}

.think-container .sub {
    color: #8899aa;
    font-size: 1rem;
    margin-bottom: 24px;
}

.input-group {
    display: flex;
    flex-direction: column;
    gap: 8px;
    margin-bottom: 16px;
}

.input-group label {
    font-size: 11px;
    letter-spacing: 1.5px;
    color: #8899aa;
    text-transform: uppercase;
    font-weight: bold;
    text-align: left;
}

.cyber-input {
    width: 100%;
    background: #020205;
    border: 1px solid #556677;
    padding: 14px 16px;
    color: #ffffff;
    font-family: 'Share Tech Mono', monospace;
    font-size: 15px;
    outline: none;
    box-shadow: inset 0 0 8px rgba(0, 255, 204, 0.05);
    transition: border-color 0.3s;
    line-height: 1.6;
    resize: vertical;
}
```

```
}
.cyber-input:focus {
  border-color: var(--primary, #00ffcc);
}

.cyber-input[readonly] {
  cursor: default;
  color: #b0c8dd;
  background: rgba(0, 0, 0, 0.4);
}

.think-actions {
  display: flex;
  gap: 16px;
  margin-top: 16px;
  flex-wrap: wrap;
}

.action-btn {
  padding: 10px 28px;
  background: transparent;
  color: var(--primary, #00ffcc);
  border: 2px solid var(--primary, #00ffcc);
  border-radius: 4px;
  font-family: 'Share Tech Mono', monospace;
  font-size: 0.95rem;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 1.5px;
  cursor: pointer;
  transition: all 0.25s ease;
  box-shadow: 0 0 12px rgba(0, 255, 204, 0.15);
  text-decoration: none;
  display: inline-block;
}

.action-btn:hover {
  background: var(--primary, #00ffcc);
  color: #0a0a14;
  box-shadow: 0 0 30px rgba(0, 255, 204, 0.35);
  transform: scale(1.04);
}

.action-btn:disabled {
  opacity: 0.4;
  cursor: not-allowed;
  transform: none !important;
}

.back-link {
  display: inline-block;
  margin-top: 24px;
  color: #8899aa;
  text-decoration: none;
  font-size: 14px;
  letter-spacing: 1px;
  transition: color 0.3s;
}
```

```

.back-link:hover {
  color: var(--primary, #00ffcc);
  text-shadow: 0 0 8px var(--primary, #00ffcc);
}

.control-panel-box-tag {
  position: absolute;
  top: -10px;
  left: 20px;
  background: #0a0a14;
  padding: 0 8px;
  font-size: 10px;
  color: var(--primary, #00ffcc);
  font-weight: bold;
  letter-spacing: 1px;
}

#status-log {
  background: rgba(0, 0, 0, 0.4);
  border: 1px dashed rgba(255, 0, 127, 0.2);
  padding: 12px;
  font-size: 12px;
  color: #8899aa;
  margin-top: 16px;
  min-height: 40px;
  line-height: 1.4;
  font-family: 'Share Tech Mono', monospace;
}

@media (max-width: 600px) {
  .think-container { padding: 20px; }
}
</style>
</head>
<body>
  <header>
    <div class="logo">
      <a href="home.html"></a>
    </div>
    <nav>
      <ul>
        <li><a href="home.html">Home</a></li>
        <li><a href="about.html">About</a></li>
        <li><a href="services.html">Services</a></li>
        <li><a href="contact.html">Contact</a></li>
      </ul>
    </nav>
  </header>

  <main class="content-page">
    <div class="think-container">
      <div class="control-panel-box-tag">🤖 THINK_NODE</div>
      <h1>Neural Query</h1>
      <div class="sub">Ask anything - the MATRIX will respond.</div>

      <!-- Question Input -->
      <div class="input-group">

```

```

        <label for="questionInput">Your Question</label>
        <textarea id="questionInput" class="cyber-input" placeholder="Ask anything..." rows="3"></textarea>
    </div>

    <!-- Answer Output (read-only) -->
    <div class="input-group">
        <label for="answerOutput">MATRIX Response</label>
        <textarea id="answerOutput" class="cyber-input" readonly placeholder="Awaiting your query ..."
rows="5"></textarea>
    </div>

    <div class="think-actions">
        <button class="action-btn" id="thinkBtn" onclick="think()">🧠 Think</button>
        <button class="action-btn" onclick="clearAll()">🗑️ Clear</button>
    </div>

    <div id="status-log">Awaiting your neural input ...</div>

    <a href="home.html" class="back-link">🏠 Return to Dashboard</a>
</div>
</main>

<script>
    // — Theme loader —
    (function applySavedTheme() {
        const theme = localStorage.getItem('vexatron_theme') || 'dark';
        document.body.className = document.body.className
            .split(' ')
            .filter(c => !c.startsWith('theme-'))
            .join(' ');
        if (theme !== 'dark') {
            document.body.classList.add('theme-' + theme);
        }
    })();

    // — ★ EMBEDDED DEEPSEEK API KEY ★ —
    // IMPORTANT: Replace this with your actual DeepSeek API key from platform.deepseek.com
    const EMBEDDED_API_KEY = 'sk-126d59a99b504a52a588b09f48389cc7';

    // — DOM refs —
    const questionInput = document.getElementById('questionInput');
    const answerOutput = document.getElementById('answerOutput');
    const statusLog = document.getElementById('status-log');
    const thinkBtn = document.getElementById('thinkBtn');

    // — Get the effective API key —
    function getApiKey() {
        // First check localStorage (in case user saved a key previously)
        const stored = localStorage.getItem('deepseek_api_key');
        if (stored && stored.startsWith('sk-')) {
            return stored;
        }
        // Fallback to embedded key
        return EMBEDDED_API_KEY;
    }

    // — Check if key is valid (not placeholder) —

```

```

function isValid(key) {
    return key && key.startsWith('sk-') && !key.includes('xxxxxxx');
}

// — Local fallback responses —
function getFallbackResponse(question) {
    const replies = [
        'The matrix acknowledges your query. Awaiting further input.',
        'I am processing your thought. Please allow a moment.',
        'there seems to be a problem... please wait while I contact VEXATRON™ self repair system'
    ];
    return replies[Math.floor(Math.random() * replies.length)];
}

// — Main Think function —
async function think() {
    const question = questionInput.value.trim();
    if (!question) {
        statusLog.style.color = '#ff5f56';
        statusLog.innerHTML = '⚠ [ERROR]: Please enter a question.';
        return;
    }

    const apiKey = getApiKey();
    if (!isValid(apiKey)) {
        statusLog.style.color = '#ff5f56';
        statusLog.innerHTML = '⚠ [ERROR]: No valid DeepSeek API key. Please replace the placeholder in the code (EMBEDDED_API_KEY) with your real key.';
        return;
    }

    thinkBtn.disabled = true;
    thinkBtn.textContent = '🧠 Thinking...';

    answerOutput.value = '';
    statusLog.style.color = 'var(--primary, #00ffcc)';
    statusLog.innerHTML = '🧠 [THINKING]: Querying DeepSeek AI...';

    try {
        const response = await fetch('https://api.deepseek.com/v1/chat/completions', {
            method: 'POST',
            headers: {
                'Content-Type': 'application/json',
                'Authorization': `Bearer ${apiKey}`
            },
            body: JSON.stringify({
                model: 'deepseek-chat',
                messages: [
                    {
                        role: 'system',
                        content: 'You are a thoughtful, insightful AI assistant called VEXATRON Matrix. You provide clear, concise, and meaningful answers. Keep responses under 200 words unless the question requires more depth.'
                    },
                    { role: 'user', content: question }
                ],
                temperature: 0.7,
            })
        });
    }

```

```

        max_tokens: 500
    })
});

if (!response.ok) {
    const errorData = await response.json().catch(() => ({}));
    const errorMsg = errorData.error?.message || `HTTP ${response.status}`;
    throw new Error(errorMsg);
}

const data = await response.json();
const aiReply = data.choices?.[0]?.message?.content || 'No response generated.';
answerOutput.value = aiReply;
statusLog.style.color = '#8899aa';
statusLog.innerHTML = '✓ [SUCCESS]: DeepSeek response received.';

} catch (error) {
    console.error('DeepSeek API error:', error);
    const fallback = getFallbackResponse(question);
    answerOutput.value = fallback;
    statusLog.style.color = '#ff5f56';
    statusLog.innerHTML = `⚠ [ERROR]: ${error.message} || 'API request failed'. Using local
fallback.`;
}

thinkBtn.disabled = false;
thinkBtn.textContent = '🧠 Think';
}

// — Clear all fields —
function clearAll() {
    questionInput.value = '';
    answerOutput.value = '';
    statusLog.style.color = '#8899aa';
    statusLog.innerHTML = '🧼 [CLEAR]: All fields reset.';
}

// — Enter key support —
questionInput.addEventListener('keydown', function(e) {
    if (e.key === 'Enter' && !e.shiftKey) {
        e.preventDefault();
        think();
    }
});
</script>
</body>
</html>

```